

Entanglement-getemperte Zeitstempel-Spektroskopie: Quaternionisches EABC-Modell und der Bamberger Zeitwürfel

(Aus dem Python-Modul `Jitter Zeit.py`)

15. Mai 2026

Zusammenfassung

Wir beschreiben ein diskretes stochastisches Zeitmodell („Zeitwürfel“), in dem Verschränkung die Streuung der Zeitintervalle und eine effektive „Vierlingsbreite“ steuert. Zeitstempel werden modulo zwölf in vier aktive Zustände E, A, B, C und einen Ruhezustand X klassifiziert und als reelle Quaternionen in die Basis $\{1, i, j, k\}$ eingebettet. Zwischen aufeinanderfolgenden EABC-Ereignissen werden relative Drehungen („Schwung“), Übergangsmatrizen und quaternionische Spektren mit komplexem Phasengewicht $e^{-i\Omega\Delta t}$ ausgewertet. Der Text folgt der Implementierung und dient als begleitende Dokumentation; er erhebt keinen Anspruch auf physikalische Ableitung aus Erstprinzipien.

1 Motivation und Überblick

Das Modell kombiniert drei Bausteine:

1. **Zufällige Zeitintervalle** Δt_k um einen durch Temperatur T und Verschränkungsparameter S bestimmten Mittelwert, mit gaußischem Jitter.
2. **EABC-Klassifikation** ganzer Zahlen $n_k \approx \alpha t_k$ modulo 12 mit Zuordnung zu Quaternion-Basisvektoren.
3. **Spektralanalyse** der vier Kanäle $(q_k)_e, (q_k)_a, (q_k)_b, (q_k)_c$ als gewichtete Fourier-Summen über Δt_k (bzw. aktive Zeiträume).

2 Quaternionische Darstellung der EABC-Zustände

Sei $\mathbb{H}$ die Quaternionenalgebra mit Hamilton-Produkt, $i^2 = j^2 = k^2 = ijk = -1$. Wir identifizieren die vier aktiven diskreten Zustände mit der

Orthonormalbasis in $\mathbb{R}^4 \cong \mathbb{H}$:

$$E \leftrightarrow 1, \quad A \leftrightarrow \mathbf{i}, \quad B \leftrightarrow \mathbf{j}, \quad C \leftrightarrow \mathbf{k}. \quad (1)$$

Der Zustand X (nicht in $\{1, 5, 7, 11\}$ modulo 12, siehe unten) wird als Null-quaternion abgebildet.

3 Klassifikation modulo 12

Für $n \in \mathbb{Z}$ sei $r = n \bmod 12$.

$$\text{label}(n) = \begin{cases} E, & r = 1, \\ A, & r = 5, \\ B, & r = 7, \\ C, & r = 11, \\ X, & \text{sonst.} \end{cases} \quad (2)$$

In der Simulation wird $n_k = \text{round}(\alpha t_k)$ gesetzt; α ist ein freier Skalenparameter (`alpha_scale`).

4 Verschränkungsabhängiger Jitter und effektive Breite

4.1 Jitter-Standardabweichung

Mit Verschränkungsparameter $S > 0$, Boden $\sigma_{\min}$, Amplitude σ_0 und Dämpfung $\gamma > 0$:

$$\sigma_t(S) = \sigma_{\min} + \sigma_0 e^{-\gamma S}. \quad (3)$$

4.2 Effektive Vierlingsbreite Δ_{eff}

Glatte Variante:

$$\Delta_{\text{eff}}(T, S) = 8 + 2(1 - e^{-T/(2\pi)})(1 - e^{-\lambda S}), \quad (4)$$

$\lambda > 0$. Grenzfälle: für $T \rightarrow 0$ gilt $\Delta_{\text{eff}} \rightarrow 8$; für große T, S nähert sich Δ_{eff} dem Wert 10.

Alternative („schärfer“) Temperaturabhängigkeit mit $\beta > 1$:

$$\Delta_{\text{eff}}^{(\beta)}(T, S) = 8 + 2(1 - e^{-\beta T/(2\pi)})(1 - e^{-\lambda S}). \quad (5)$$

5 Zeitwürfel-Simulation

Gegeben $\omega_0 > 0$ und Δ_{eff} wie oben setzt die Implementierung

$$\Delta t^{(0)} = \frac{\Delta_{\text{eff}}}{\omega_0}. \quad (6)$$

Für $k = 0, \dots, N-1$:

$$\varepsilon_k \sim \mathcal{N}(0, \sigma_t^2), \quad \Delta t_k = \max\{\Delta t^{(0)} + \varepsilon_k, \Delta t^{(0)}\}, \quad (7)$$

$$t_{k+1} = t_k + \Delta t_k, \quad t_0 = 0. \quad (8)$$

Zu jedem Schritt werden n_k , $\text{label}(n_k)$ und die Quaternion-Komponenten von q_k gespeichert.

Für zwei Quaternion-Zustände q_k, q_{k+1} der *vollen* Simulation definiert der Code den relativen Schwung

$$u_k := \bar{q}_k q_{k+1} \in \mathbb{H}, \quad (9)$$

mit Konjugation $\bar{q}$ und Hamilton-Produkt. Die Komponenten $(u_k)_e, (u_k)_a, (u_k)_b, (u_k)_c$ und $\|u_k\|$ werden protokolliert.

6 Aktive EABC-Folge und Übergänge

Alle Zeilen mit $\text{label} \in \{E, A, B, C\}$ bilden die **aktive Folge**. Zwischen aufeinanderfolgenden aktiven Ereignissen werden Zeiten t_k und

$$\Delta t_{\text{active},k} := t_k - t_{k-1} \quad (10)$$

(wie in der Implementierung: erste Zeile **NaN**) verwendet.

Nur zwischen aufeinanderfolgenden *aktiven* Quaternionen wird erneut der Schwung

$$u = \bar{q}_k^{(\text{akt})} q_{k+1}^{(\text{akt})} \quad (11)$$

gebildet.

6.1 Schwung-Label

Sei $u = u_e + u_a i + u_b j + u_c k$. Das Label wählt den Index E, A, B, C mit maximalem $|\cdot|$ unter den reellen Koeffizienten und ein Vorzeichen:

$$\text{schwung_label} \in \{0, +E, -A, \dots\}. \quad (12)$$

6.2 Übergangsmatrix und Idealrotation

Die Matrix der Paare $(\text{label}_{\text{from}}, \text{label}_{\text{to}})$ wird auf feste Zeilen und Spalten E, A, B, C expandiert (fehlende Einträge gleich null).

Als **Idealrotation** zählt die Implementierung die Übergänge

$$A \rightarrow E, \quad E \rightarrow C, \quad C \rightarrow B, \quad B \rightarrow A; \quad (13)$$

ihr relativer Anteil an allen aktiven Übergängen wird ausgegeben (**Idealrotations-Anteil**).

Algorithm 1 Ablauf der Zeitwürfel-Simulation und nachgeschaltete Auswertung (Δ_{eff} hier mit Temperaturfaktor β wie in `delta_eff_sharp`; in der Implementierung wird Δt_k zusätzlich nach unten durch $\Delta_{\text{eff}}/\omega_0$ begrenzt, falls ε_k zu negativ ausfällt). Die kompakte Schreibweise $\mathcal{P}_{\mathbb{H}}$ entspricht im Code der Summe der kanalweisen Leistungen $|Q_X|^2$, $X \in \{e, a, b, c\}$, vgl. Abschnitt ??.

- 1: Wähle $N, T, S, \omega_0, \sigma_{\min}, \sigma_0, \gamma, \lambda, \beta$.
- 2: Berechne den entanglement-abhängigen Jitter

$$\sigma_t(S) = \sigma_{\min} + \sigma_0 e^{-\gamma S}.$$

- 3: Berechne die effektive Breite

$$\Delta_{\text{eff}}(T, S) = 8 + 2(1 - e^{-\beta T/(2\pi)})(1 - e^{-\lambda S}).$$

- 4: **for** $k = 1, \dots, N$ **do**
- 5: Ziehe $\varepsilon_k \sim \mathcal{N}(0, \sigma_t^2)$.
- 6: Setze

$$\Delta t_k = \Delta_{\text{eff}}/\omega_0 + \varepsilon_k.$$

- 7: Setze

$$t_k = \sum_{j \leq k} \Delta t_j.$$

- 8: Runde $n_k = \lfloor \alpha t_k \rfloor$ und klassifiziere $n_k \bmod 12$.
- 9: Weise

$$E \mapsto 1, \quad A \mapsto \text{i}, \quad B \mapsto \text{j}, \quad C \mapsto \text{k}$$

zu (Einbettung von $q_k \in \mathbb{H}$).

- 10: **end for**
- 11: Filtere die aktive EABC-Folge.
- 12: Berechne den Schwung $u_k = \overline{q_k} q_{k+1}$.
- 13: Berechne das Spektrum

$$\mathcal{P}_{\mathbb{H}}(\Omega) = \left| \sum_k q_k e^{-i\Omega \Delta t_k} \right|^2.$$

7 Quaternionisches Spektrum

Sei $\Omega \in \mathbb{R}$ eine Kreisfrequenz und Δt_k die gewählte Intervallfolge (z. B. Simulationsintervall oder Δt_{active}). Mit Phasen $\phi_k(\Omega) = e^{-i\Omega\Delta t_k}$ definieren wir komponentenweise

$$Q_X(\Omega) := \sum_k (q_k)_X \phi_k(\Omega), \quad X \in \{e, a, b, c\}, \quad (14)$$

und die Leistungen $P_X(\Omega) = |Q_X(\Omega)|^2$ sowie

$$P_{\text{ges}}(\Omega) = P_E + P_A + P_B + P_C. \quad (15)$$

Das entspricht einer komponentenweisen komplexen Fourier-Summe mit Gewichtung durch die Zeitintervalle (nicht durch gleichmäßige Abtastung in t).

8 Zentriertes Spektrum

Um den Gleichanteil der Kanäle zu unterdrücken, ersetzt die Implementierung

$$\tilde{q}_k := q_k - \bar{q}, \quad \bar{q}_X := \frac{1}{N'} \sum_k (q_k)_X, \quad (16)$$

(N' Anzahl der verwendeten Zeilen nach Filter) und wendet dieselbe Spektralformel auf $\tilde{q}$ an. Optional wird nur die aktive Teilfolge verwendet; für Δt kann man Δt_k der Simulation oder Δt_{active} wählen (`dt_column`).

9 Rigidity-Proxy und Vergleichsfolgen

Die Datei enthält weiterhin einen einfachen Autokorrelations-Proxy für normierte Intervallfluktuationen sowie Hilfsfunktionen zum Vergleich zweier Intervallstatistiken (vgl. Nullstellenabstände, Primlücken usw.). Details sind der Implementierung zu entnehmen.

10 Implementierung

Die beschriebenen Objekte entsprechen den Funktionen und Klassen in `JitterZeit.py`, insbesondere: `simulate_zeitwuerfel`, `quaternionic_spectrum`, `active_eabc_sequence`, `active_transition_schwung`, `transition_matrix`, `centered_quaternionic_spectrum`, `print_transition_report`.

Hinweis

Das Modell ist als exploratives und reproduzierbares Rechenexperiment gedacht. Physikalische Interpretation von T, S, Ω und der EABC-Zuordnung bleibt dem Nutzer überlassen.